

Hopfield Networks — это все, что вам нужно

Пост в блоге

[Посмотреть на GitHub](#)

В этом посте рассказывается о статье [Hopfield Networks is All You Need](#) и соответствующем новом слое PyTorch [Hopfield layer](#).

Оглавление

1. Основные достижения
2. О чем этот пост в блоге
3. От классических сетей Хопфилда к самовниманию
 1. Сети Хопфилда
 2. Современные сети Хопфилда
 3. Новая энергетическая функция для непрерывных паттернов и состояний
 4. Обновление новой энергетической функции — это механизм внутреннего внимания в трансформаторных сетях
4. Слои Хопфилда для архитектур глубокого обучения
 1. Слой *Hopfield*
 2. Слой *HopfieldLayer*
 3. Поиск по алгоритму Хопфилда через *HopfieldLayer*
 4. Слой *HopfieldPooling*
5. DeepRC
6. Материал
7. Переписка

Основные результаты

Мы вводим новую функцию энергии и соответствующее новое правило обновления, которое гарантированно приводит к локальному минимуму функции энергии.

Новая энергетическая функция является обобщением (дискретные состояния $\Rightarrow$ непрерывные состояния) **современных сетей Хопфилда**, также известных как **плотные ассоциативные воспоминания**, были представлены [Кротовым и Хопфилдом](#) и [Демирджигилом и др.](#) Новая современная сеть Хопфилда с непрерывными состояниями сохраняет характеристики своих дискретных аналогов:

- экспоненциальная емкость хранилища
- чрезвычайно быстрая сходимость

Благодаря непрерывным состояниям эта новая современная сеть Хопфилда является дифференцируемой и может быть интегрирована в архитектуры глубокого обучения. Как правило, паттерны восстанавливаются после одного обновления, что позволяет активировать слои глубоких сетей. Это открывает широкие возможности для создания **новых архитектур глубокого обучения**. Предлагаются три полезных типа слоев Хопфилда.

Удивительно, но новое правило обновления — это механизм внимания в трансформерных сетях, представленный в статье [Attention Is All You Need](#). Мы используем эти новые идеи для анализа трансформерных моделей в нашей статье.

О чем этот пост в блоге

Этот пост в блоге разбит на три части. Во-первых, мы рассказываем о переходе от традиционных сетей Хопфилда к **современным сетям Хопфилда** и их обобщении на непрерывные состояния с помощью нашей **новой энергетической функции**. Во-вторых, мы демонстрируем свойства нашей новой энергетической функции и ее связь с механизмом внутреннего внимания в трансформерных сетях. Наконец, мы представляем и объясняем работу нового слоя PyTorch (**слоя Хопфилда**), созданного на основе наших идей. Мы приводим несколько практических примеров использования, в том числе [современные сети Хопфилда и механизм внимания для классификации иммунного репертуара](#), объединение Хопфилдом и сопоставление двух наборов.

От классических сетей Хопфилда до механизма внутреннего внимания

Ассоциативная память — одна из самых ранних искусственных нейронных моделей, появившихся в 1960–1970-х годах. Наиболее известны [сети Хопфилда](#), представленные Джоном Хопфилдом в 1982 году. Как следует из названия, основная задача сетей ассоциативной памяти — связывать входные данные с наиболее похожими на них шаблонами. Другими словами, их задача — хранить и извлекать шаблоны. Начнём с обзора классических сетей Хопфилда.

Сети Хопфилда

Самая простая ассоциативная память — это просто **сумма внешних произведений** N шаблонов $\{\mathbf{x}_i\}_{i=1}^N$ которые мы хотим сохранить (правило обучения Хебба). В классических сетях Хопфилда эти паттерны являются полярными (бинарными), то есть $\mathbf{x}_i \in \{-1, 1\}^d$, где d — длина узоров. Соответствующая весовая матрица $\mathbf{W}$ является:

$$\mathbf{W} = \sum_i^N \mathbf{x}_i \mathbf{x}_i^T. \quad (1)$$

The weight matrix $\mathbf{W}$ stores the patterns, which can be retrieved starting with a **state pattern** ξ .

Nomenclature

From now on we denote the N **stored patterns** as $\{\mathbf{x}_i\}_{i=1}^N$ and any **state pattern** or **state** as $\boldsymbol{\xi}$.

The basic **synchronous update rule** is to repeatedly multiply the state pattern $\boldsymbol{\xi}$ with the weight matrix $\mathbf{W}$, subtract the bias and take the sign:

$$\boldsymbol{\xi}^{t+1} = \text{sgn}(\mathbf{W}\boldsymbol{\xi}^t - \mathbf{b}) , \quad (2)$$

where $\mathbf{b} \in \mathbb{R}^d$ is a bias vector, which can be interpreted as threshold for every component. The **asynchronous update rule** performs this update only for one component of $\boldsymbol{\xi}$ and then selects the next component for update. Convergence is reached if $\boldsymbol{\xi}^{t+1} = \boldsymbol{\xi}^t$.

The asynchronous version of the update rule of Eq. (2) minimizes the **energy function** E :

$$E = -\frac{1}{2}\boldsymbol{\xi}^T \mathbf{W} \boldsymbol{\xi} + \boldsymbol{\xi}^T \mathbf{b} = -\frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d w_{ij} \xi_i \xi_j + \sum_{i=1}^d b_i \xi_i . \quad (3)$$

As derived in the papers of [Bruck](#), [Goles-Chacc et al.](#) and [the original Hopfield paper](#), the convergence properties are dependent on the structure of the weight matrix $\mathbf{W}$ and the method by which the nodes are updated:

- For asynchronous updates with $w_{ii} \geq 0$ and $w_{ij} = w_{ji}$, the updates converge to a stable state.
- For synchronous updates with $w_{ij} = w_{ji}$, the updates converge to a stable state or a limit cycle of length 2.

For the asynchronous update rule and symmetric weights, $E(\boldsymbol{\xi}^{t+1}) \leq E(\boldsymbol{\xi}^t)$ holds. When $E(\boldsymbol{\xi}^{t+1}) = E(\boldsymbol{\xi}^t)$ for the update of every component of $\boldsymbol{\xi}^t$, a local minimum in E is reached. All stored patterns $\{\mathbf{x}_i\}_{i=1}^N$ should be fixed points of the Hopfield Network, i.e.

$$\mathbf{x}_i = \text{sgn}(\mathbf{W}\mathbf{x}_i - \mathbf{b}) . \quad (4)$$

They should even be local minima of E .

In the following example, no bias vector is used. This means that taking the inverse image, i.e. flipping all pixels at once, results in the same energy.

We start with an **illustrative example** of a Hopfield Network. **One input image** should first be stored and then be retrieved. The input image is:



Since an associative memory has **polar states and patterns** (or binary states and patterns), we convert the input image to a black and white image:



The **weight matrix** $\mathbf{W}$ is the outer product of this black and white image $\mathbf{x}_{\text{Homer}}$:

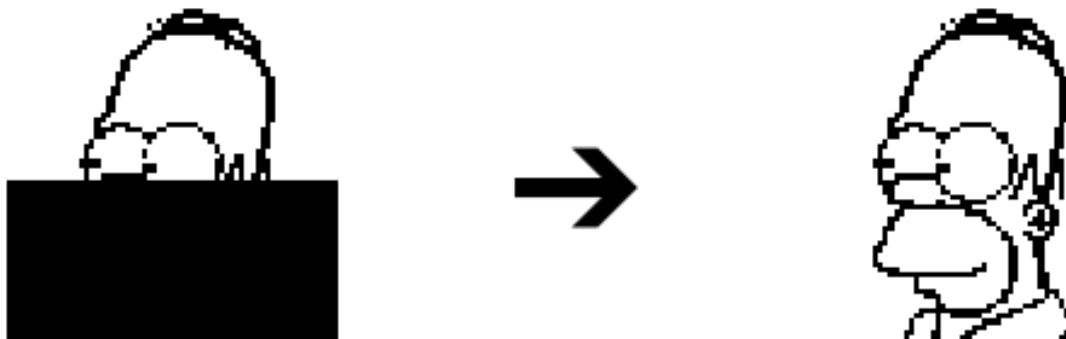
$$\mathbf{W} = \mathbf{x}_{\text{Homer}} \mathbf{x}_{\text{Homer}}^T, \quad \mathbf{x}_{\text{Homer}} \in \{-1, 1\}^d, \quad (5)$$

where for this example $d = 64 \times 64$.

Can the original image be restored if half of the pixels are masked out? The masked image is:



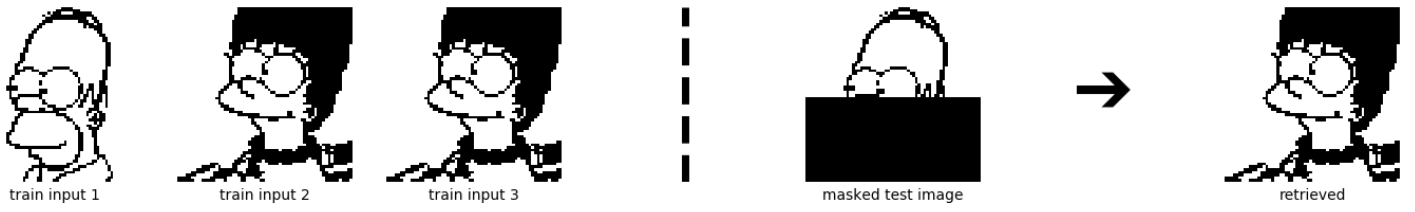
which is our initial state ξ . This initial state is updated via multiplication with the weight matrix $\mathbf{W}$. It takes one update until the original image is restored.



What happens if we store **more than one pattern**? The weight matrix is then built from the sum of outer products of **three stored patterns** (three input images):

$$\mathbf{W} = \sum_{i=1}^3 \mathbf{x}_i \mathbf{x}_i^T, \quad \mathbf{x}_i \in \{-1, 1\}^d. \quad (6)$$



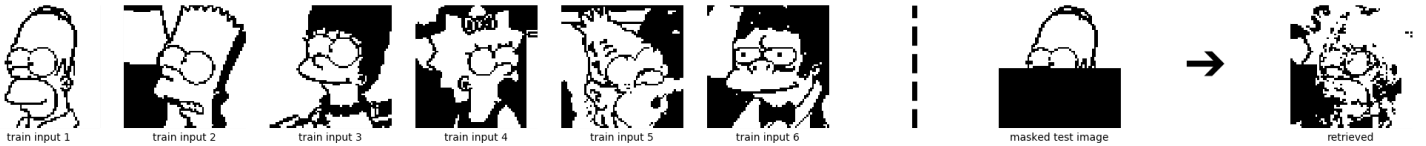


In this figure, the left hand side shows the three stored patterns, and the right hand side shows masked state patterns ξ together with the retrieved patterns ξ^{new} .

Looking at the upper row of images might suggest that the retrieval process is no longer perfect. But there are two interesting facts to take into account:

- Masking the original images introduces many pixel values of -1 . We therefore have the odd behavior that the inner product $\langle \mathbf{x}_{\text{Homer}}^{\text{masked}}, \mathbf{x}_{\text{Bart}} \rangle$ is larger than the inner product $\langle \mathbf{x}_{\text{Homer}}^{\text{masked}}, \mathbf{x}_{\text{Homer}} \rangle$.
- As stated above, if no bias vector is used, the inverse of the pattern, i.e. flipping all pixels at once, results in the same energy.

Although the retrieval of the upper image looks incorrect, it is de facto correct. However, for the lower row example, the retrieval is no longer correct. The weights of $2 \cdot \mathbf{x}_{\text{Marge}}$ have simply overwritten the weights of $\mathbf{x}_{\text{Homer}}$. For both examples, only the retrieval after the first update step is shown, but the results do not change when performing further update steps. The next figure shows the Hopfield Network retrieval for 6 patterns.



Clearly, retrieving the patterns is imperfect. One might suspect that the limited storage capacities of Hopfield Networks, see [Amit et al.](#) and [Torres et al.](#), is the problem. However, we show now that the storage capacity is not directly responsible for the imperfect retrieval. The storage capacity for **retrieval of patterns free of errors** is:

$$C \cong \frac{d}{2 \log(d)}, \quad (7)$$

where d is the dimension of the input.

The storage capacity for **retrieval of patterns with a small percentage of errors** is:

$$C \cong 0.14d. \quad (8)$$

In the example, the storage capacity is $C \cong 0.14d = 0.14 \cdot 64 \cdot 64 \sim 570$. Thus, insufficient storage capacity is not directly responsible for the retrieval errors. Instead, the example patterns are correlated, therefore the retrieval has errors.

Consequently, we need a model which **allows pulling apart close patterns**, such that (strongly) **correlated patterns can be distinguished**.

On storage capacity

The storage capacities stated in Eq. (7) and in Eq. (8) are derived for $w_{ii} = 0$. Recently, [Folli et al.](#) analyzed the storage capacity for Hopfield Networks with $w_{ii} \geq 0$. Also for $w_{ii} \geq 0$, a storage capacity of $C \cong 0.14d$ for retrieval of patterns with a small percentage of errors was observed. The ratio C/d is often called **load parameter** and denoted by α . [Folli et al.](#) showed that there is a second regime with very large α , where the storage capacity is much higher, i.e. more fixed points exist. However, [Rocci et al.](#) and [Gosti et al.](#) reported that these fixed points for very large α are unstable and do not have an attraction basin.

Modern Hopfield Networks (aka Dense Associative Memories)

The storage capacity is a crucial characteristic of Hopfield Networks. **Modern Hopfield Networks** (aka Dense Associative Memories) introduce a new energy function instead of the energy in Eq. (3) to create a higher storage capacity. **Discrete** modern Hopfield Networks have been introduced first by [Krotov and Hopfield](#) and then generalized by [Demircigil et al.](#):

- [Krotov and Hopfield](#) introduced the energy function:

$$E = - \sum_{i=1}^N F(\mathbf{x}_i^T \boldsymbol{\xi}) , \quad (9)$$

where F is an interaction function and N is again the number of stored patterns.

They choose a polynomial interaction function $F(z) = z^a$.

The storage capacity for **retrieval of patterns free of errors** is:

$$C \cong \frac{1}{2(2a-3)!!} \frac{d^{a-1}}{\log(d)} . \quad (10)$$

The storage capacity for **retrieval of patterns with a small percentage of errors** is:

$$C \cong \alpha_a d^{a-1} , \quad (11)$$

where α_a is a constant, which depends on an (arbitrary) threshold on the error probability.

For $a = 2$, the classical Hopfield model (Hopfield 1982) is obtained with the storage capacity of $C \cong 0.14d$ for retrieval of patterns with a small percentage of errors.

- [Demircigil et al.](#) extended the energy function by using an exponential interaction function $F(z) = \exp(z)$:

$$E = - \sum_{i=1}^N \exp(\mathbf{x}_i^T \boldsymbol{\xi}) , \quad (12)$$

where N is again the number of stored patterns.

Eq. (12) can also be written as:

$$E = -\exp(\text{lse}(1, \mathbf{X}^T \boldsymbol{\xi})) , \quad (13)$$

where $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$ is the data matrix (matrix of stored patterns).

The *log-sum-exp function* (lse) is defined as:

$$\text{lse}(\beta, \mathbf{z}) = \beta^{-1} \log \left(\sum_{l=1}^N \exp(\beta z_l) \right) . \quad (14)$$

This energy function leads to the storage capacity:

$$C \cong 2^{\frac{d}{2}} . \quad (15)$$

We now look at the **update rule**, which is valid for both Eq. (9) as well as Eq. (12). For polar patterns, i.e. $\boldsymbol{\xi} \in \{-1, 1\}^d$, we denote the l -th component by $\boldsymbol{\xi}[l]$. Using the energy function of Eq. (9) or Eq. (12), the update rule for the l -th component $\boldsymbol{\xi}[l]$ is described by the difference of the energy of the current state $\boldsymbol{\xi}$ and the state with the component $\boldsymbol{\xi}[l]$ flipped. The component $\boldsymbol{\xi}[l]$ is updated to decrease the energy. The update rule is:

$$\boldsymbol{\xi}^{\text{new}}[l] = \text{sgn} \left[-E(\boldsymbol{\xi}^{(l+)}) + E(\boldsymbol{\xi}^{(l-)}) \right] , \quad (16)$$

which is (e.g. for Eq. (12))

$$\boldsymbol{\xi}^{\text{new}}[l] = \text{sgn} \left[\sum_{i=1}^N \exp(\mathbf{x}_i^T \boldsymbol{\xi}^{(l+)}) - \sum_{i=1}^N \exp(\mathbf{x}_i^T \boldsymbol{\xi}^{(l-)}) \right] , \quad (17)$$

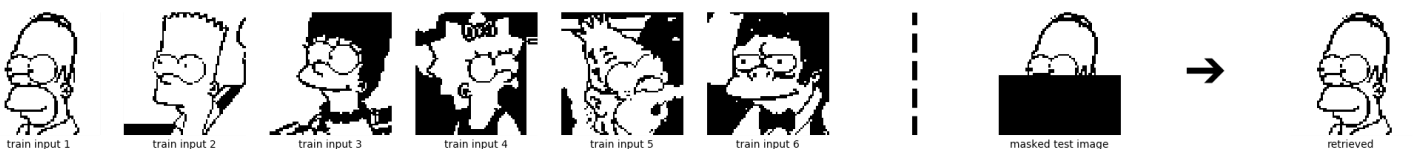
where $\boldsymbol{\xi}^{(l+)}[l] = 1$ and $\boldsymbol{\xi}^{(l-)}[l] = -1$ and $\boldsymbol{\xi}^{(l+)}[k] = \boldsymbol{\xi}^{(l-)}[k] = \boldsymbol{\xi}[k]$ for $k \neq l$.

In the paper of [Demircigil et al.](#), it is shown that the **update rule**, which minimizes the energy function of Eq. (12), converges with high probability after one (asynchronous) update of the current state $\boldsymbol{\xi}$. Note that one update of the current state $\boldsymbol{\xi}$ corresponds to d asynchronous update steps, i.e. one update for each of the d single components $\boldsymbol{\xi}[l]$ ($l = 1, \dots, d$).

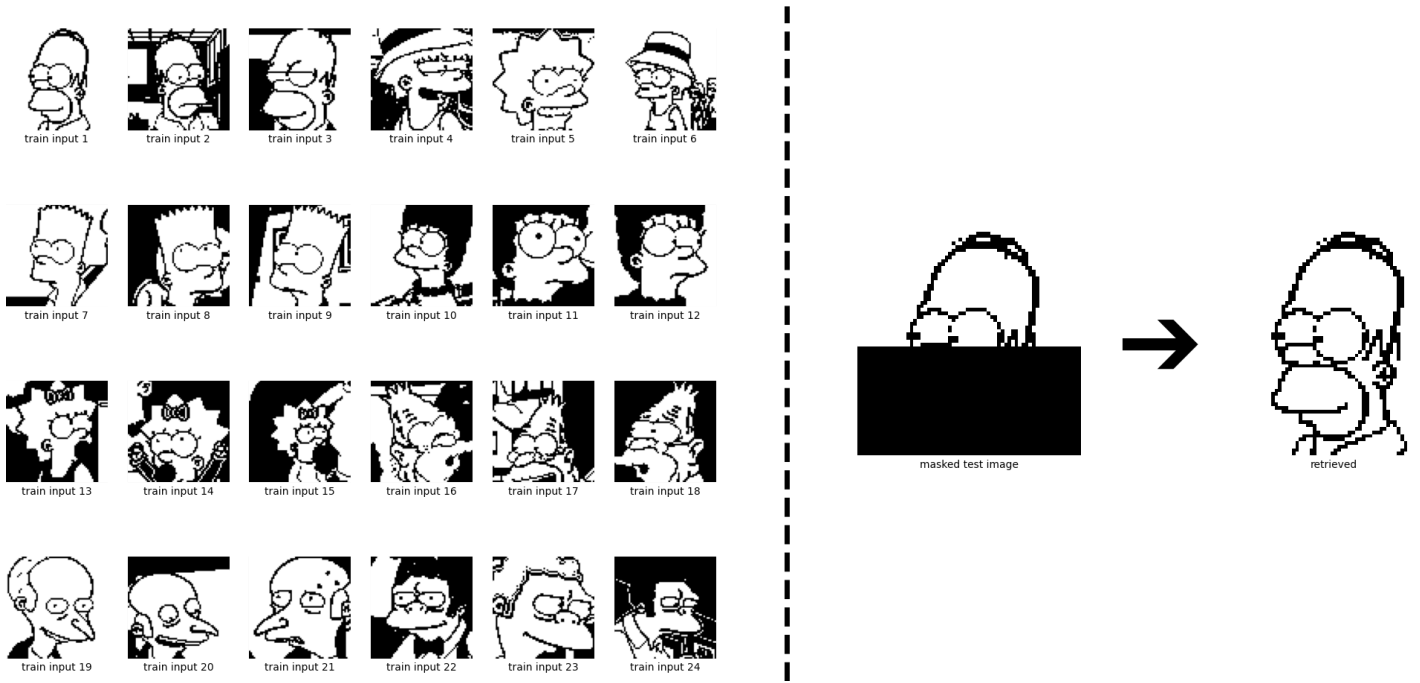
In contrast to classical Hopfield Networks, modern Hopfield Networks do not have a weight matrix as it is defined in Eq. (5). Instead, the energy function is the **sum of a function of the dot product** of every stored pattern $\mathbf{x}_i$ with the state pattern $\boldsymbol{\xi}$.

Using Eq. (17), we again try to retrieve Homer out of the 6 stored patterns. Compared to the classical Hopfield Network, it now works smoothly, not only for 6 patterns but also for many more:

- First we store the same 6 patterns as above:



- Next we increase the number of stored patterns to 24:



Compared to the traditional Hopfield Networks, the **increased storage capacity now allows pulling apart close patterns**. We are now able to distinguish (strongly) correlated patterns, and can retrieve one specific pattern out of many.

New energy function for continuous-valued patterns and states

We generalize the energy function of Eq. (13) to continuous-valued patterns. We use the logarithm of the negative energy Eq. (13) and add a quadratic term. The quadratic term ensures that the norm of the state ξ remains finite. The **new energy function** is defined as:

$$E = -\text{lse}(\beta, \mathbf{X}^T \xi) + \frac{1}{2} \xi^T \xi + \beta^{-1} \log N + \frac{1}{2} M^2, \quad (18)$$

which is constructed from N **continuous** stored patterns by the matrix $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$, where M is the largest norm of all stored patterns.

According to the [new paper of Krotov and Hopfield](#), the stored patterns $\mathbf{X}^T$ of our modern Hopfield Network can be viewed as weights from ξ to hidden units, while $\mathbf{X}$ can be viewed as weights from the hidden units to ξ . With this interpretation we do not store patterns, but use only weights in our model as in the classical Hopfield Network.

The energy function of Eq. (18) allows deriving an update rule for a state pattern ξ by the **Concave-Convex-Procedure** (CCCP), which is described by [Yuille and Rangarajan](#).

- the total energy $E(\xi)$ is split into a convex and a concave term: $E(\xi) = E_1(\xi) + E_2(\xi)$
- the term $\frac{1}{2} \xi^T \xi + C = E_1(\xi)$ is convex (C is a constant independent of ξ)
- the term $-\text{lse}(\beta, \mathbf{X}^T \xi) = E_2(\xi)$ is concave (lse is convex since its Hessian is positive semi-definite, which is shown in the appendix of the paper)
- The CCCP applied to E is:

$$\nabla_{\xi} E_1(\xi^{t+1}) = -\nabla_{\xi} E_2(\xi^t) \quad (19)$$

$$\nabla_{\xi} \left(\frac{1}{2} \xi^T \xi + C \right) (\xi^{t+1}) = \nabla_{\xi} \text{lse}(\beta, \mathbf{X}^T \xi^t) \quad (20)$$

$$\xi^{t+1} = \mathbf{X} \text{softmax}(\beta \mathbf{X}^T \xi^t), \quad (21)$$

where $\nabla_{\xi} \text{lse}(\beta, \mathbf{X}^T \xi) = \mathbf{X} \text{softmax}(\beta \mathbf{X}^T \xi)$.

The update rule for a state pattern ξ therefore reads:

$$\xi^{\text{new}} = \mathbf{X} \text{softmax}(\beta \mathbf{X}^T \xi). \quad (22)$$

Having applied the Concave-Convex-Procedure to obtain the update rule guarantees the monotonical decrease of the energy function.

The most important **properties of our new energy function** are:

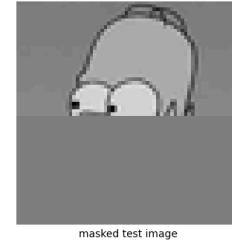
1. Global convergence to a local minimum (Theorem 2 in the paper)
2. Exponential storage capacity (Theorem 3 in the paper)
3. Convergence after one update step (Theorem 4 in the paper)

Exponential storage capacity and convergence after one update are inherited from [Demircigil et al.](#) Global convergence to a local minimum means that all limit points that are generated by the iteration of Eq. (22) are stationary points (local minima or saddle points) of the energy function of Eq. (18) (almost surely no maxima are found, saddle points were never encountered in any experiment).

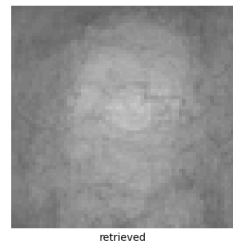
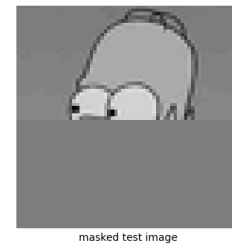
The new continuous energy function allows **extending our example to continuous patterns**. In the following, we are going to retrieve a continuous Homer out of many continuous stored patterns using Eq. (22). First we have to convert the input images into grey scale images:



Next, we conduct the same experiment as above, but now in continuous form:



We again see that Homer is perfectly retrieved. We have considered the case where the patterns are sufficiently different from each other, and consequently the iterate converges to a fixed point which is near to one of the stored patterns. However, if some stored patterns are similar to each other, then a **metastable state** near the similar patterns appears. Iterates that start near this metastable state or at one of the similar patterns converge to this metastable state. The **learning dynamics can be controlled by the inverse temperature β** , see Eq. (22). High values of β correspond to a low temperature and mean that the attraction basins of the individual patterns remain separated and it is unlikely that metastable states appear. Low values of β on the other hand correspond to a high temperature and the formation of metastable states becomes more likely. We now look at the same example, but instead of $\beta = 8$, we use $\beta = 0.5$. The retrieved state is now a superposition of multiple stored patterns.



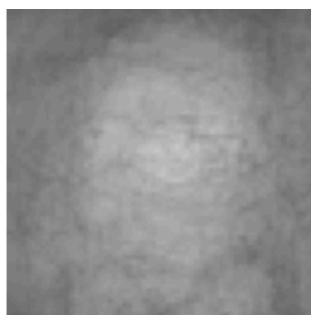
To make this more explicit, we have a closer look how the results are changing if we retrieve with different values of β :



masked



$\beta = 0.25$



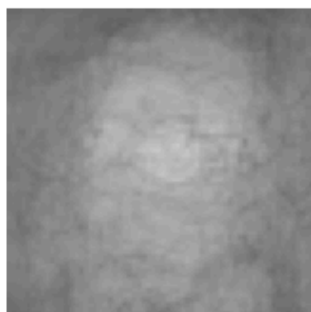
retrieved



masked



$\beta = 0.50$



retrieved



masked



$\beta = 1.00$



retrieved



masked



$\beta = 2.00$



retrieved



masked



$\beta = 4.00$



retrieved





masked

beta = 8.00



retrieved

The update of the new energy function is the self-attention of transformer networks

Starting with Eq. (21), and

1. generalizing the new update rule to multiple patterns at once,
2. mapping the patterns to an associative space,
3. projecting the result,

we arrive at the **(self-)attention of transformer networks**. Next, we will guide through these three steps.

For S state patterns $\Xi = (\xi_1, \dots, \xi_S)$, Eq. (21) can be generalized to:

$$\Xi^{\text{new}} = \mathbf{X} \text{softmax}(\beta \mathbf{X}^T \Xi) . \quad (23)$$

We first consider $\mathbf{X}^T$ as N raw stored patterns $\mathbf{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_N)^T$, which are mapped to an associative space via $\mathbf{W}_K$, and Ξ^T as S raw state patterns $\mathbf{R} = (\xi_1, \dots, \xi_S)^T$, which are mapped to an associative space via $\mathbf{W}_Q$.

Setting

$$\mathbf{Q} = \Xi^T = \mathbf{R} \mathbf{W}_Q , \quad (24)$$

$$\mathbf{K} = \mathbf{X}^T = \mathbf{Y} \mathbf{W}_K , \quad (25)$$

$$\beta = \frac{1}{\sqrt{d_k}} , \quad (26)$$

we obtain:

$$(\mathbf{Q}^{\text{new}})^T = \mathbf{K}^T \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{K} \mathbf{Q}^T \right) . \quad (27)$$

In Eq. (24) and Eq. (25), $\mathbf{W}_Q$ and $\mathbf{W}_K$ are matrices which map the respective patterns into the associative space. Note that in Eq. (27), the softmax is applied column-wise to the matrix $\mathbf{K} \mathbf{Q}^T$.

Next, we simple transpose Eq. (27), which also means that the softmax is now applied row-wise to its transposed input $\mathbf{Q} \mathbf{K}^T$, and obtain:

$$\mathbf{Q}^{\text{new}} = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^T \right) \mathbf{K} . \quad (28)$$

Now, we only need to project $\mathbf{Q}^{\text{new}}$ via another projection matrix $\mathbf{W}_V$:

$$\mathbf{Z} = \mathbf{Q}^{\text{new}} \mathbf{W}_V = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^T \right) \mathbf{K} \mathbf{W}_V = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^T \right) \mathbf{V}, \quad (29)$$

and **voilà, we have obtained the transformer attention**. If the N raw stored patterns $\mathbf{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_N)^T$ are used as raw state patterns $\mathbf{R}$, we obtain the **transformer self-attention**.

If we resubstitute our raw stored patterns $\mathbf{Y}$ and our raw state patterns $\mathbf{R}$, we can rewrite Eq. (29) as

$$\mathbf{Z} = \text{softmax}(\beta \cdot \mathbf{R} \mathbf{W}_Q \mathbf{W}_K^T \mathbf{Y}^T) \mathbf{Y} \mathbf{W}_K \mathbf{W}_V, \quad (30)$$

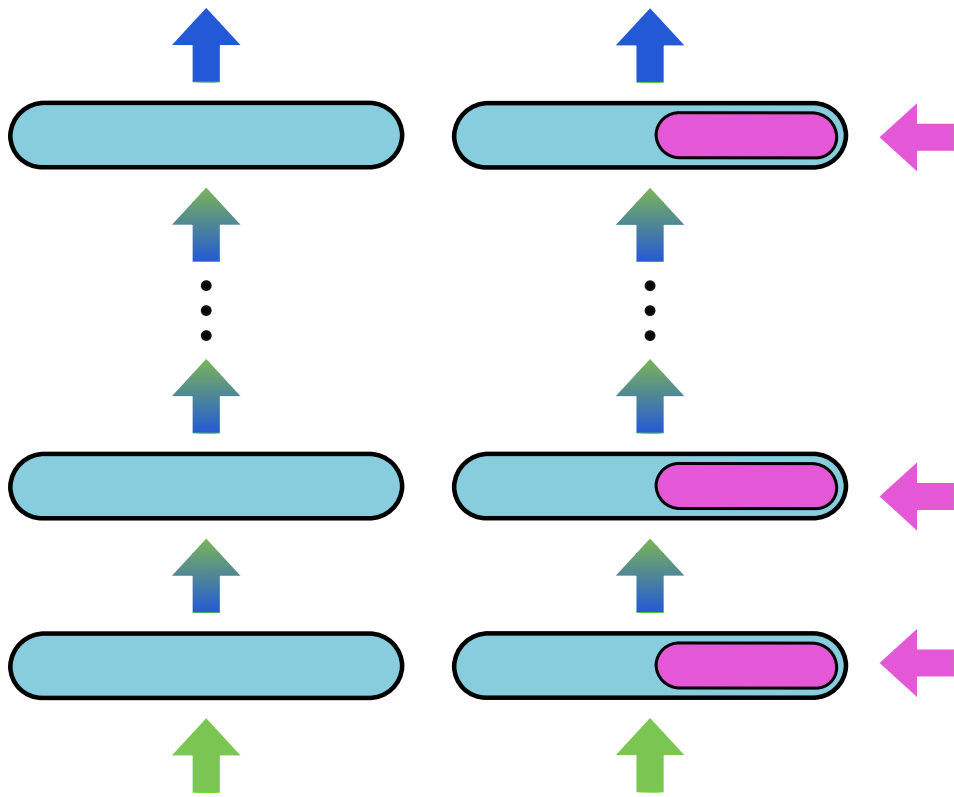
which is the fundament of our new PyTorch Hopfield layer.

Hopfield layers for Deep Learning architectures

The insights stemming from our work on modern Hopfield Networks allowed us to introduce new [PyTorch Hopfield layers](#), which can be used as plug-in replacement for existing layers as well as for applications like multiple instance learning, set-based and permutation invariant learning, associative learning, and many more. We introduce three types of Hopfield layers:

- `Hopfield` for associating and processing two sets. Examples are the transformer attention, which associates keys and queries, and two point sets that have to be compared.
- `HopfieldPooling` for fixed pattern search, pooling operations, and memories like LSTMs or GRUs. The state (query) pattern is static and can be learned.
- `HopfieldLayer` for storing fixed patterns or learning internal prototypes. The stored (key) patterns are static and can be learned.

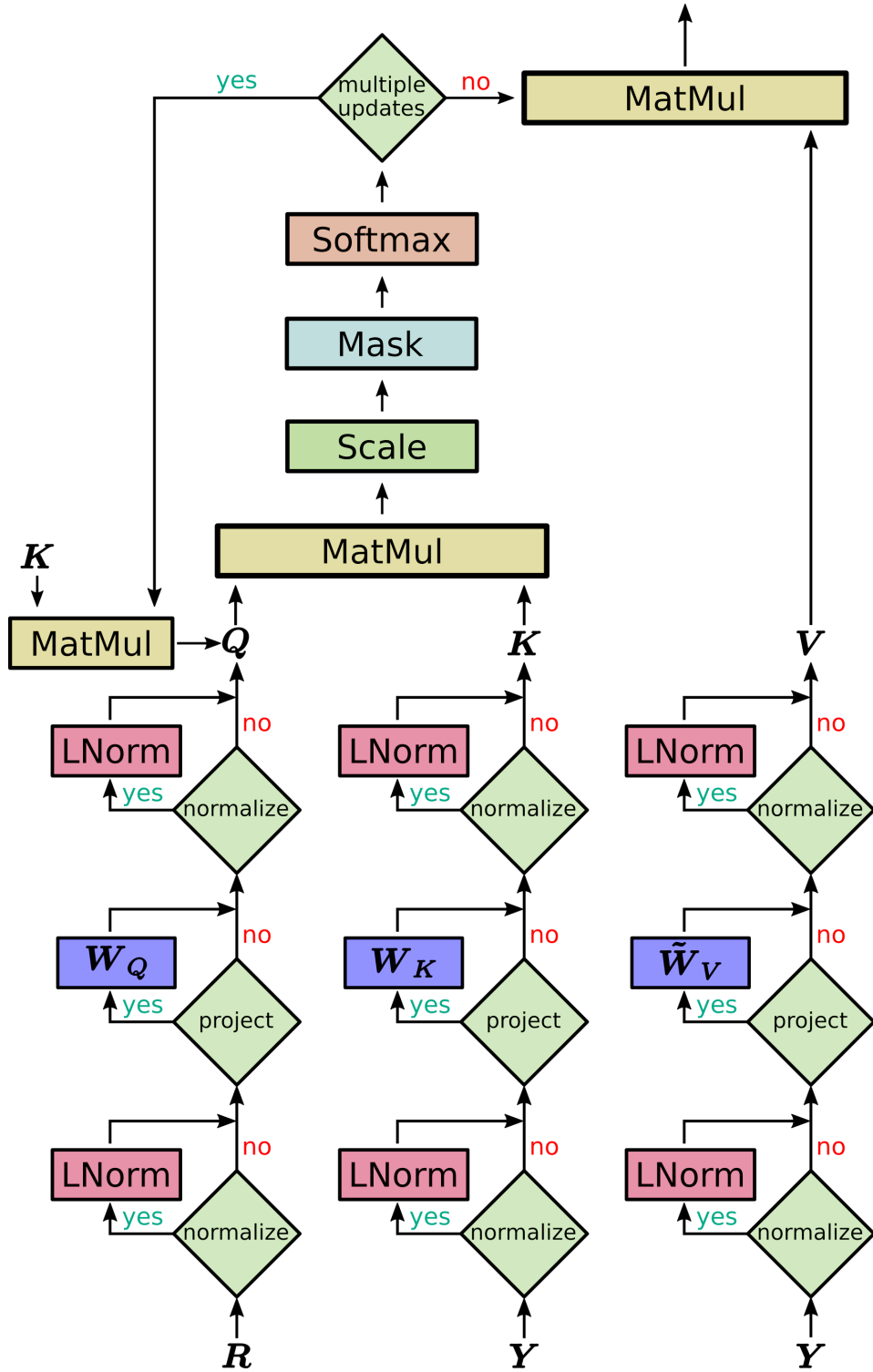
Due to their continuous nature Hopfield layers are differentiable and can be integrated into deep learning architectures to equip their layers with associative memories. On the left side of the Figure below a standard deep network is depicted. It propagates either a vector or a set of vectors from input to output. On the right side a deep network is depicted, where layers are equipped with associative memories via Hopfield layers. A detailed description of the layers is given below.



Additional functionalities of the new PyTorch Hopfield layers compared to the transformer (self-)attention layer are:

- **Association of two sets**
- **Variable** β that determines the kind of fixed points
- **Multiple updates** for precise fixed points
- **Dimension of the associative space** for controlling the storage capacity
- **Static patterns** for fixed pattern search
- **Pattern normalization** to control the fixed point dynamics by norm and shift of the patterns

A sketch of the new Hopfield layers is provided below.



Next, we introduce the underlying mechanisms of the implementation. Based on these underlying mechanisms, we give three examples on how to use the new Hopfield layers and how to utilize the principles of modern Hopfield Networks.

Layer Hopfield

In its most general form, the result patterns $\mathbf{Z}$ are a function of raw stored patterns $\mathbf{Y}$, raw state patterns $\mathbf{R}$, and projection matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$:

$$\mathbf{Z} = f(\mathbf{Y}, \mathbf{R}, \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V) = \text{softmax}(\beta \cdot \mathbf{R} \mathbf{W}_Q \mathbf{W}_K^T \mathbf{Y}^T) \mathbf{Y} \mathbf{W}_K \mathbf{W}_V \quad (31)$$

where we denote

$$\tilde{\mathbf{W}}_V = \mathbf{W}_K \mathbf{W}_V. \quad (32)$$

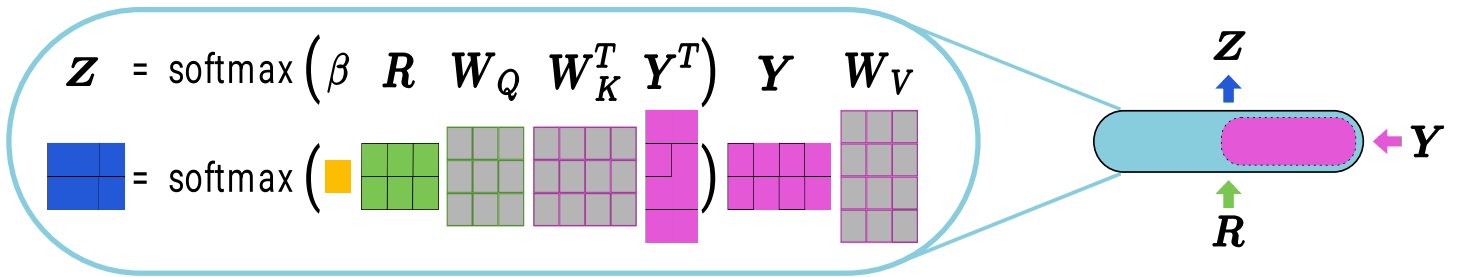
Here, the rank of $\tilde{\mathbf{W}}_V$ is limited by dimension constraints of the matrix product $\mathbf{W}_K \mathbf{W}_V$. To provide the Hopfield layer with more flexibility, the matrix product $\mathbf{W}_K \mathbf{W}_V$ can be replaced by one parameter matrix (flag in the code). In this case $\tilde{\mathbf{W}}_V$ is not the product from Eq. (32) but a stand-alone parameter matrix as in the original transformer setting.

In Eq. (31), the N raw stored patterns $\mathbf{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_N)^T$ and the S raw state patterns $\mathbf{R} = (\mathbf{r}_1, \dots, \mathbf{r}_S)^T$ are mapped to an associative space via the matrices $\mathbf{W}_K$ and $\mathbf{W}_Q$. We also allow **static state** and **static stored patterns**. A static pattern means that it does not depend on the network input, i.e. it is determined by the bias weights and remains constant across different network inputs.

Remark

For simplicity from now on we replace $\mathbf{W}_K \mathbf{W}_V$ by just $\mathbf{W}_V$.

An illustration of the matrices of Eq. (31) is shown below:



Note that in this simplified sketch $\mathbf{W}_V$ already contains the output projection.

It now depends on the underlying tasks which matrices are used. For example, the code for the above sketch would be the following:

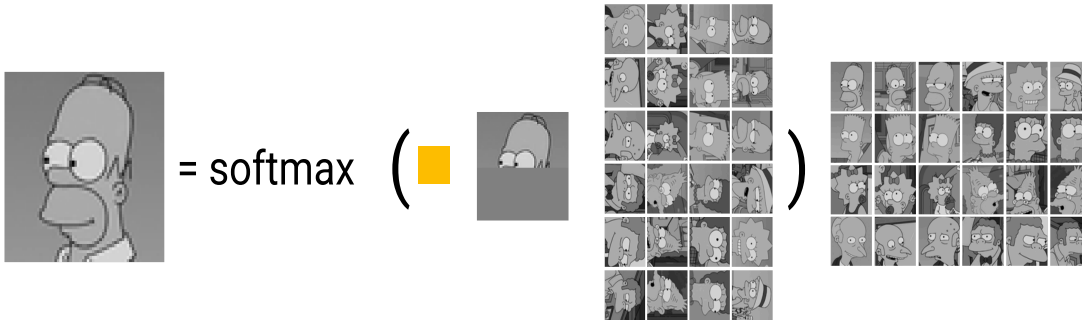
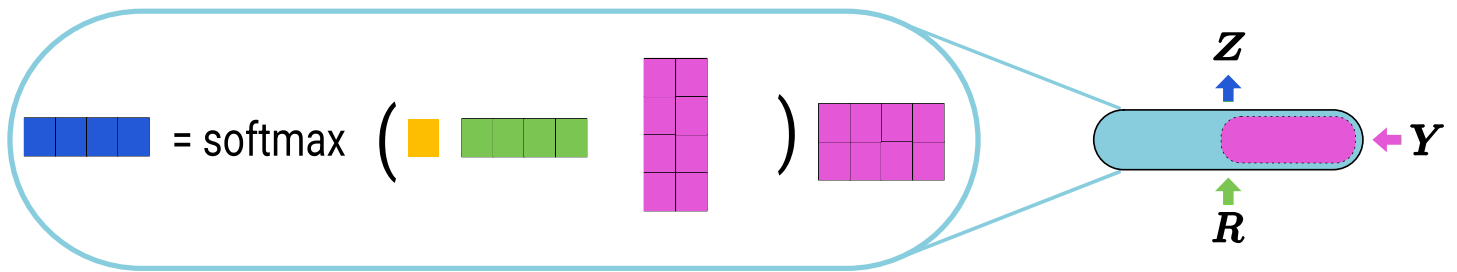
```
hopfield = Hopfield(
    input_size=3,                    # R
    hidden_size=3,
    stored_pattern_size=4,           # Y
    pattern_projection_size=4,       # Y
    scaling=beta)

# tuple of stored_pattern, state_pattern, pattern_projection
hopfield((Y, R, Y))
```

Layer HopfieldLayer

Of course we can also use the new Hopfield layer to solve the pattern retrieval task from above. For this task no trainable weights are needed.

$$\mathbf{Z} = \text{softmax} \left(\beta \mathbf{R} \mathbf{Y}^T \right) \mathbf{Y}$$



```
hopfield = Hopfield(
    scaling=beta,

    # do not project layer input
    state_pattern_as_static=True,
    stored_pattern_as_static=True,
    pattern_projection_as_static=True,

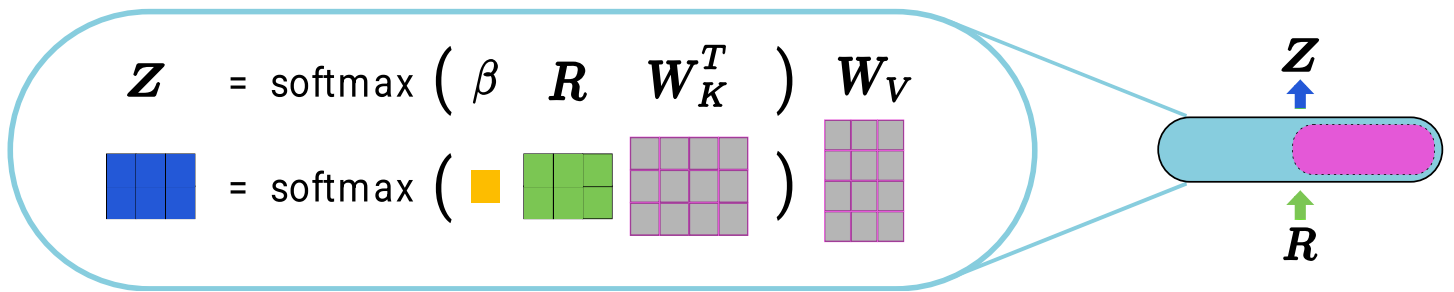
    # do not pre-process layer input
    normalize_stored_pattern=False,
    normalize_stored_pattern_affine=False,
    normalize_state_pattern=False,
    normalize_state_pattern_affine=False,
    normalize_pattern_projection=False,
    normalize_pattern_projection_affine=False,

    # do not post-process layer output
    disable_out_projection=True)

# tuple of stored_pattern, state_pattern, pattern_projection
hopfield((Y, R, Y))
```

Hopfield Lookup via *HopfieldLayer*

A variant of our Hopfield-based modules is one which employs a **trainable but input independent lookup mechanism**. Internally, one or multiple **stored patterns and pattern projections are trained** (optionally in a non-shared manner), which in turn are used as a lookup mechanism independent of the input data.



```
hopfield_lookup = HopfieldLayer(
    input_size=3,                    # R
    hidden_size=3,                   # W_K
    pattern_size=4,                  # W_V
    quantity=4,                      # W_K
    scaling=beta,
    stored_pattern_as_static=True,
    state_pattern_as_static=True)

# state pattern
hopfield_lookup(R)
```

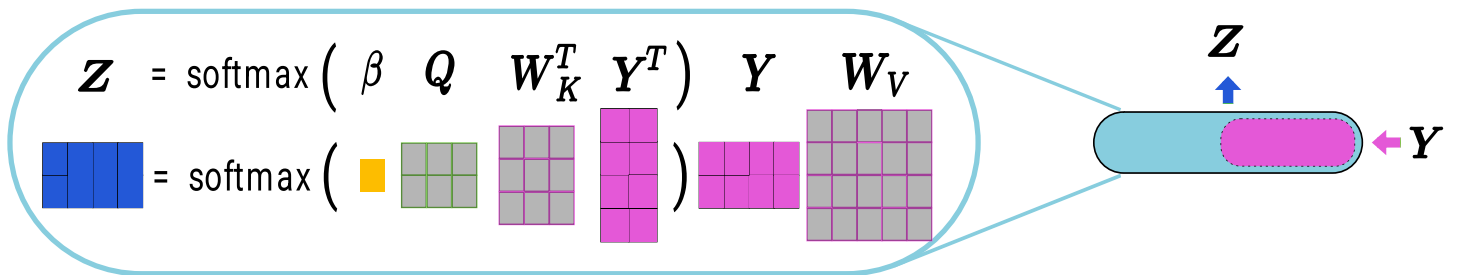
This specialized variant of the Hopfield layer allows for a setting, where the training data is used as stored patterns, the new data as state pattern, and the training label to project the output of the Hopfield layer.

```
hopfield_lookup = HopfieldLayer(
    input_size=3,                    # R
    hidden_size=3,                   # W_K
    quantity=4,                      # W_K
    scaling=beta,
    lookup_weights_as_separated=True,
    lookup_targets_as_trainable=False,
    stored_pattern_as_static=True,
    state_pattern_as_static=True,
    pattern_projection_as_static=True)

# state pattern
hopfield_lookup(R)
```

Layer HopfieldPooling

We consider the Hopfield layer as a pooling layer if only one static state pattern (query) exists. Then, it is de facto a **pooling over the sequence**. The static state pattern is considered as a **prototype pattern** and consequently learned in the Hopfield pooling layer. Below we give two examples of a Hopfield pooling over the stored patterns $\mathbf{Y}$. Note that the pooling always operates over the token dimension (i.e. the sequence length), and not the token embedding dimension.

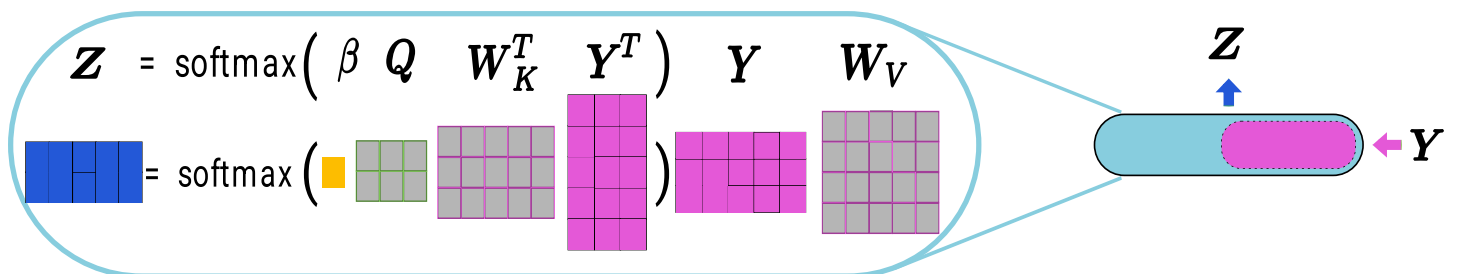


```
hopfield_pooling = HopfieldPooling(
    input_size=4,                    # Y
    hidden_size=3,                  # Q
    scaling=beta,
    quantity=2)                    # number of state patterns

# stored_pattern and pattern_projection
hopfield_pooling(Y)
```

The pooling over the sequence is de facto done over the token dimension of the stored patterns, i.e. $\mathbf{Y} \in \mathbb{R}^{(2 \times 4)} \Rightarrow \mathbf{Z} \in \mathbb{R}^{(2 \times 4)}$.

We show another example below, where the Hopfield pooling boils down to $\mathbf{Y} \in \mathbb{R}^{(3 \times 5)} \Rightarrow \mathbf{Z} \in \mathbb{R}^{(2 \times 5)}$:



```
hopfield_pooling = HopfieldPooling(
    input_size=5,                    # Y
    hidden_size=3,                  # Q
    scaling=beta,
    quantity=2)                    # number of state patterns

# stored_pattern and pattern_projection
hopfield_pooling(Y)
```

DeepRC

One SOTA application of modern Hopfield Networks can be found in the paper [Modern Hopfield Networks and Attention for Immune Repertoire Classification](#) by Widrich et al. Here, the high storage capacity of modern Hopfield Networks is exploited to solve a challenging [multiple instance learning \(MIL\)](#) problem in computational biology called **immune repertoire classification**.

Biological background

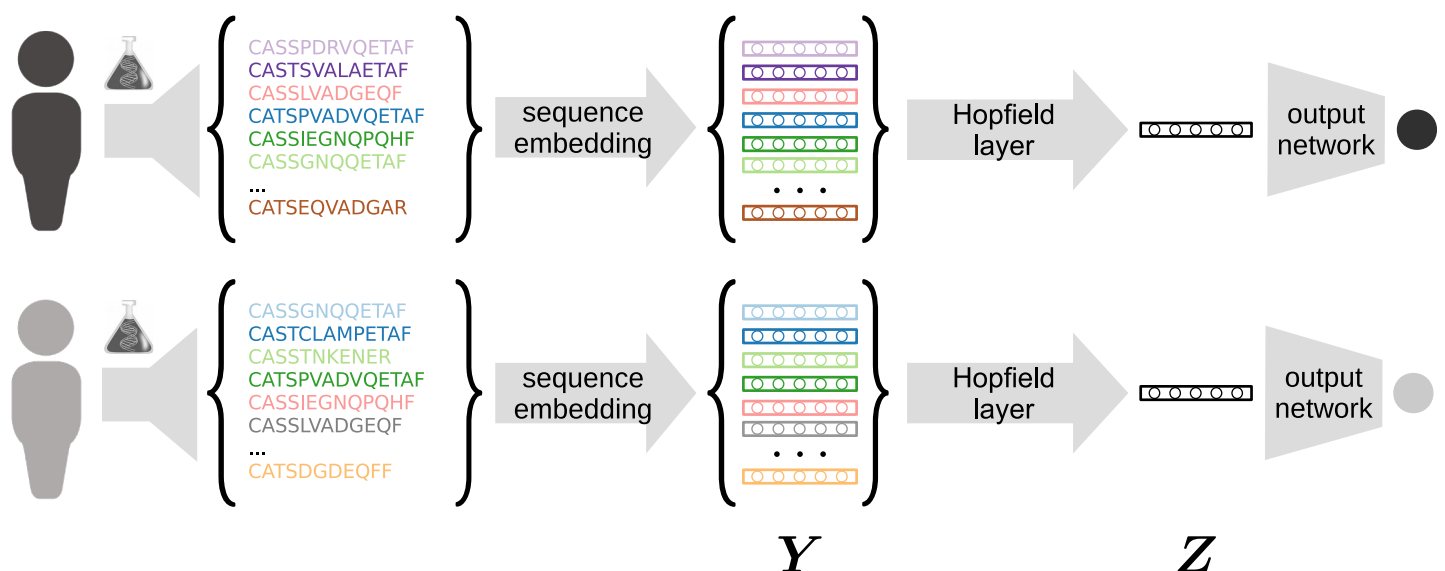
The immune repertoire of an individual consists of an immensely large number of immune repertoire receptors (and many other things). The task of these receptors, which can be represented as amino acid sequences with variable length and 20 unique letters, is to selectively bind to surface-structures of specific pathogens in order to combat them. Only a variable sub-sequence of the receptors might be responsible for this binding. Due to the large variety of pathogens, each human has about 10^7 – 10^8 unique immune receptors with low overlap across individuals and sampled from a potential diversity of $> 10^{14}$ receptors. However, only very few of these receptors bind to a single specific pathogen. This means that the immune repertoire of an individual that shows an immune response against a specific pathogen, e.g. a specific disease, should contain a few sequences that can bind to this specific pathogen. Turning this around, in order to classify such immune repertoires into those with and without immune response, one would have to find this variable sub-sequence that binds to the specific pathogen.

Consequently, the **classification of immune repertoires is extremely difficult**, since each immune repertoire contains a large amount of sequences as instances with only a very few of them indicating the correct class by carrying a certain variable sub-sequence. This is a prominent example of a **needle-in-a-haystack** problem and a strong challenge for machine learning methods.

Based on modern Hopfield networks, a method called **DeepRC** was designed, which consists of three parts:

- a sequence-embedding neural network to supply a fixed-sized sequence-representation (e.g. 1D-CNN or LSTM),
- a **Hopfield layer part** for sequence-attention, and
- an output neural network and/or fully connected output layer.

The following figure illustrates these 3 parts of DeepRC:

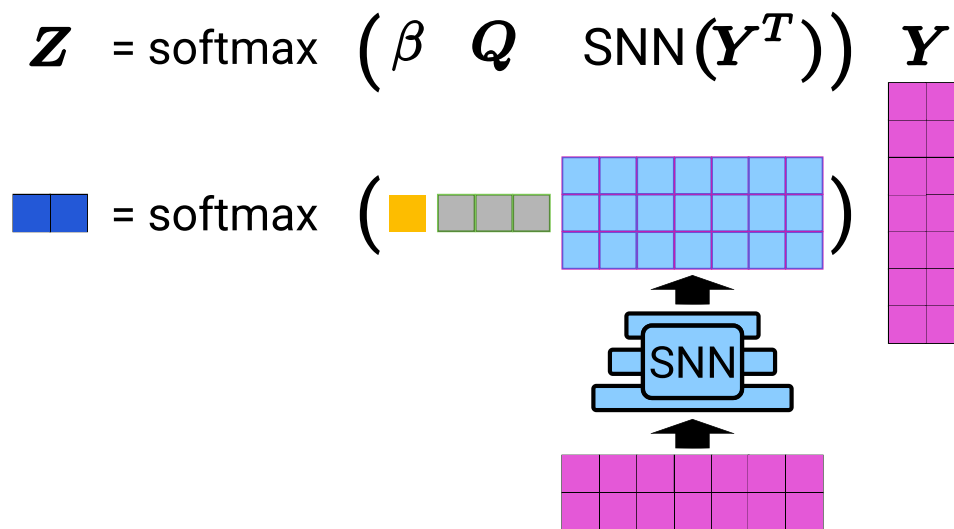


So far we have discussed two use cases of the Hopfield layer: (i) the default setting where the **input consists of stored patterns and state patterns** and (ii) the Hopfield pooling, where a **prototype pattern** is learned, which means that the vector Q is learned. For immune repertoire classification we have another use case. Now the **inputs** for the Hopfield layer are partly **obtained via neural networks**.

To be more precise, the three ingredients of the attention mechanism of **DeepRC** are:

- the output of the sequence-embedding neural network $\mathbf{Y}^T$ directly acts as values $\mathbf{V}$,
- a second neural network, e.g. a [self-normalizing neural network \(SNN\)](#), shares its first layers with the sequence-embedding neural network and outputs the keys $\mathbf{K}$, i.e. the stored patterns, and
- similar to the Hopfield pooling operation, the query vector $\mathbf{Q}$ is learned and represents the variable binding sub-sequence we are looking for.

The following sketch visualizes the **Hopfield layer part** of DeepRC:



It is to note that for immune repertoire classification the **number of instances is much larger than the number of features (~300k instances per repertoire)**. To be more precise, the difference is a factor of 10^4 to 10^5 . This is indicated in the sketch, where $\mathbf{Y}^T$ has more columns than rows. The complex SNN-based attention mechanism reduces this large number of instances, while keeping the complexity of the input to the output neural network low.

The pseudo-code for the [Hopfield layer](#) used in DeepRC is:

```
Y = EmbeddingNN(I)  # e.g. 1D-CNN
K = SNN(Y)
Q = StatePattern(size=3)

hopfield = Hopfield(
    scaling=beta,

    # do not project layer input
    state_pattern_as_static=True,
    stored_pattern_as_static=True,
    pattern_projection_as_static=True,

    # do not pre-process layer input
    normalize_stored_pattern=False,
    normalize_stored_pattern_affine=False,
    normalize_state_pattern=False,
    normalize_state_pattern_affine=False,
```

```
normalize_pattern_projection=False,  
normalize_pattern_projection_affine=False,  
  
# do not post-process layer output  
disable_out_projection=True)  
  
# tuple of stored_pattern, state_pattern, pattern_projection  
hopfield((K, Q, Y))
```

Material

- [Paper: Hopfield Networks is All You Need](#)
- [Github repo: hopfield-layers](#)
- [Paper: Modern Hopfield Networks and Attention for Immune Repertoire Classification](#)
- [Github repo: DeepRC](#)
- [Yannic Kilcher's video on our two papers](#)
- [Blog post on Performers from a Hopfield point of view](#)

For more information visit our homepage <https://ml-jku.github.io/>.

Correspondence

This blog post was written by Johannes Brandstetter: [brandstetter\[at\]ml.jku.at](mailto:brandstetter[at]ml.jku.at)

Contributions by Viet Tran, Bernhard Schäfl, Hubert Ramsauer, Johannes Lehner, Michael Widrich, Günter Klambauer and Sepp Hochreiter.

hopfield-layers is maintained by [ml-jku](#).

This page was generated by [GitHub Pages](#).